

“Prototipo software gestión de cine”

Andrés Felipe Obando Barriga y colaboradores académicos

Resumen

Se creó un prototipo de software en tres fases para optimizar la gestión de turnos y venta de boletas en salas de cine colombianas. La primera fase se centró en la lógica y programación orientada a objetos para el funcionamiento del sistema. La segunda incorporó interfaces gráficas intuitivas y solucionó problemas previos. La tercera integró una base de datos MySQL para gestión centralizada de datos y corrigió errores. Como resultado, se obtuvo un software eficiente y escalable que brinda un sistema eficiente de gestión de turnos y compra de entradas, mejorando la experiencia del cliente y los procesos operativos. Las interfaces amigables permiten a usuarios visualizar horarios, seleccionar asientos y comprar boletas de forma ágil. La base de datos garantiza un manejo organizado de la información. En conclusión, esta solución tecnológica aborda efectivamente la problemática de confusión y retrasos en cines colombianos, aumentando la satisfacción del cliente y la rentabilidad del negocio, creando un prototipo completo que funciona como base a proyectos de la misma índole más avanzados.

Abstract

A software prototype was developed in three phases to optimize the management of shifts and ticket sales in Colombian cinemas. The first phase focused on the logic and object-oriented programming for the system's functionality. The second phase incorporated intuitive graphical interfaces and addressed previous issues. The third phase integrated a MySQL database for centralized data management and corrected errors. As a result, an efficient and scalable software was obtained, providing an efficient system for shift management and ticket purchasing, enhancing customer experience and operational processes. User-friendly interfaces allow users to view schedules, select seats, and purchase tickets seamlessly. The database ensures organized handling of information. In conclusion, this technological solution effectively addresses the issues of confusion and delays in Colombian cinemas, increasing customer satisfaction and business profitability, making a full prototype that works like a base-work of an advanced project of the same subject.

1. Introduction

En la industria del entretenimiento, específicamente en el ámbito de las salas de cine, uno de los mayores desafíos que enfrentan las empresas es la gestión eficiente de los turnos y la venta de boletas. Esta problemática no solo genera confusión y retrasos, sino que también puede afectar negativamente la experiencia del cliente y la rentabilidad del negocio.

En Colombia, donde la industria del cine ha experimentado un crecimiento significativo en los últimos años, muchas salas de cine han enfrentado dificultades para mantener un sistema organizado y fluido de turnos y venta de boletas. Las largas filas, la confusión en los horarios y la falta de información precisa pueden causar frustración en los espectadores, lo que a su vez podría llevarlos a buscar alternativas de entretenimiento.

Reconociendo esta problemática, nuestro equipo se propuso crear un prototipo de software innovador que abordara estas cuestiones de manera efectiva. El proyecto se llevó a cabo en tres fases distintas, cada una enfocada en diferentes aspectos clave del software.

La primera fase se centró en la lógica y la programación orientada a objetos, estableciendo una base sólida para el funcionamiento del sistema. Se definieron las clases y objetos necesarios para representar los conceptos principales, como salas de cine, películas, horarios y boletas. Además, se implementaron algoritmos eficientes para manejar la asignación de asientos y la gestión de turnos.

En la segunda fase, se añadió la parte de interfaces gráficas de usuario (GUI), lo que permitió una interacción más intuitiva y amigable con el sistema. Se diseñaron pantallas atractivas y fáciles de usar para que los clientes pudieran ver los horarios de las películas, seleccionar sus asientos y completar la compra de boletas de manera rápida y sencilla. Además, se implementaron soluciones a los problemas identificados en la primera versión, mejorando aún más la experiencia de uso.

La tercera fase se dedicó a la resolución de errores de la fase 2 y se integró una base de datos MySQL para la gestión centralizada de toda la información relacionada con la venta de boletas y la organización de los turnos. Esta base de datos permitió almacenar y recuperar datos de manera eficiente, facilitando la gestión de la información y garantizando la integridad de los datos.

Con este prototipo de software, las salas de cine en Colombia podrán disfrutar de un sistema optimizado y eficiente para la gestión de turnos y venta de boletas. Gracias a su enfoque en la usabilidad, la escalabilidad y la robustez, el software garantizará una

experiencia fluida y sin contratiempos tanto para los espectadores como para el personal de las salas de cine, contribuyendo a mejorar la calidad del servicio, la satisfacción del cliente y, en última instancia, la rentabilidad del negocio.

2. Marco teórico

La gestión eficiente de turnos y venta de boletas en salas de cine es esencial para optimizar la experiencia del cliente y los procesos operativos. En este sentido, se requiere un enfoque sistémico y organizado que integre diversos conceptos y teorías relevantes:

- **Gestión de Turnos y Venta de Boletas:**

Se refiere a la planificación y control de los horarios de proyección de películas y la comercialización de boletas para garantizar una distribución equitativa de la capacidad de las salas y una satisfacción óptima del cliente.

- **Programación Orientada a Objetos (POO):**

Es un paradigma de programación que utiliza la abstracción y el modularidad para representar entidades del mundo real mediante clases y objetos. En el contexto del software para salas de cine, la POO facilita la representación de elementos como salas, películas, horarios y boletas, así como la gestión de su interacción.

- **Interfaces Gráficas Intuitivas:**

Son herramientas visuales diseñadas para facilitar la interacción entre el usuario y el sistema. En el contexto de la gestión de turnos y venta de boletas, las interfaces gráficas intuitivas permiten a los usuarios visualizar información relevante, como horarios de proyección y disponibilidad de asientos, de manera clara y accesible.

- **Base de Datos MySQL:**

Es un sistema de gestión de bases de datos relacional que proporciona un almacenamiento estructurado y eficiente de la información. En el contexto del software para salas de cine, MySQL se utiliza para gestionar datos relacionados con películas, horarios, disponibilidad de asientos y transacciones de compra de boletas.

- **Experiencia del Cliente y Rentabilidad del Negocio:**

Son aspectos cruciales para el éxito de una sala de cine. Una experiencia positiva del cliente, facilitada por un sistema eficiente de gestión de turnos y venta de boletas, contribuye a aumentar la satisfacción y fidelización de los clientes, así como la rentabilidad del negocio al mejorar la eficiencia operativa y maximizar la ocupación de las salas.

3. Desarrollo de ingeniería

Diagrama de casos de uso:

Casos de uso Software cine

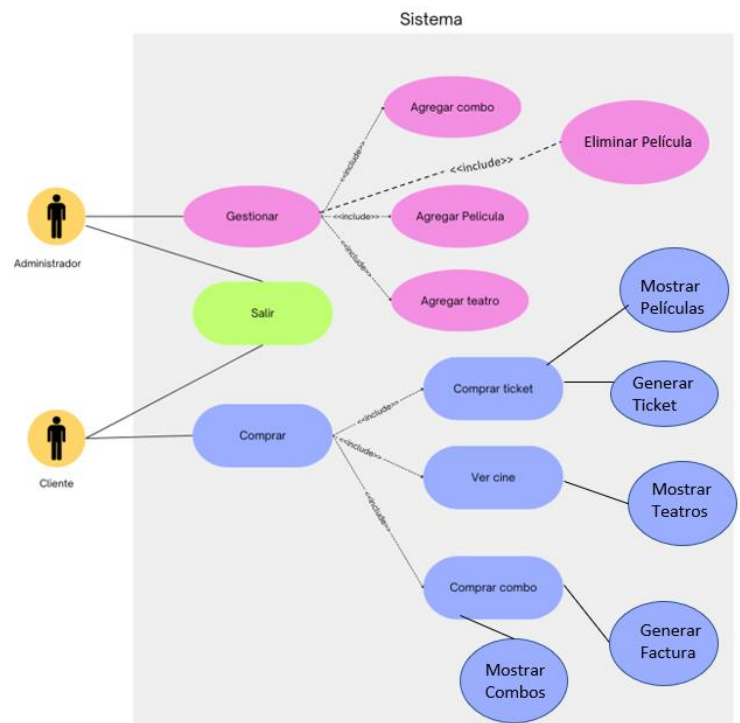


Diagrama de clases UML:

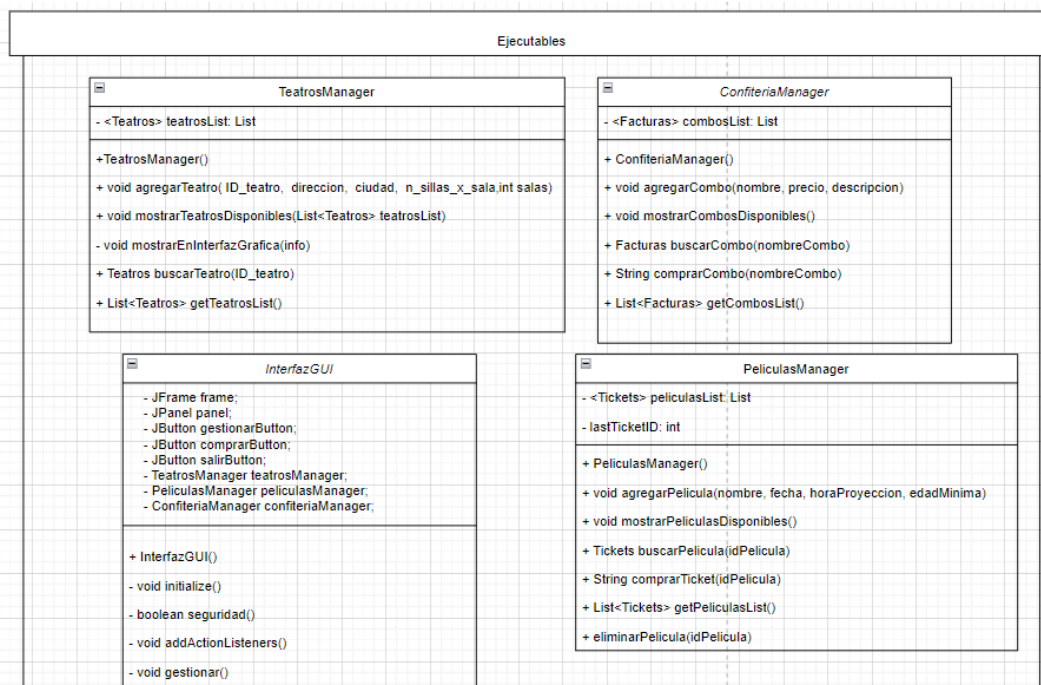
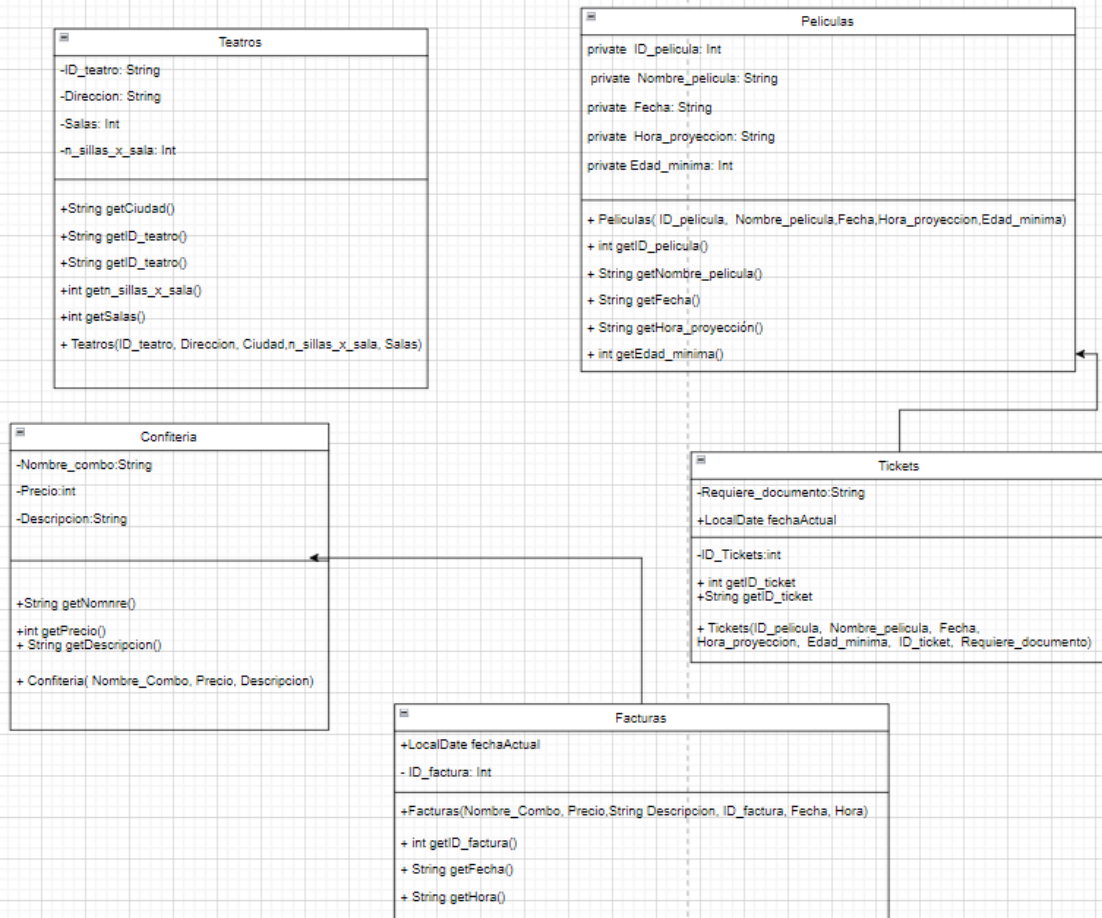


Diagrama de entidad relación para la base de datos:

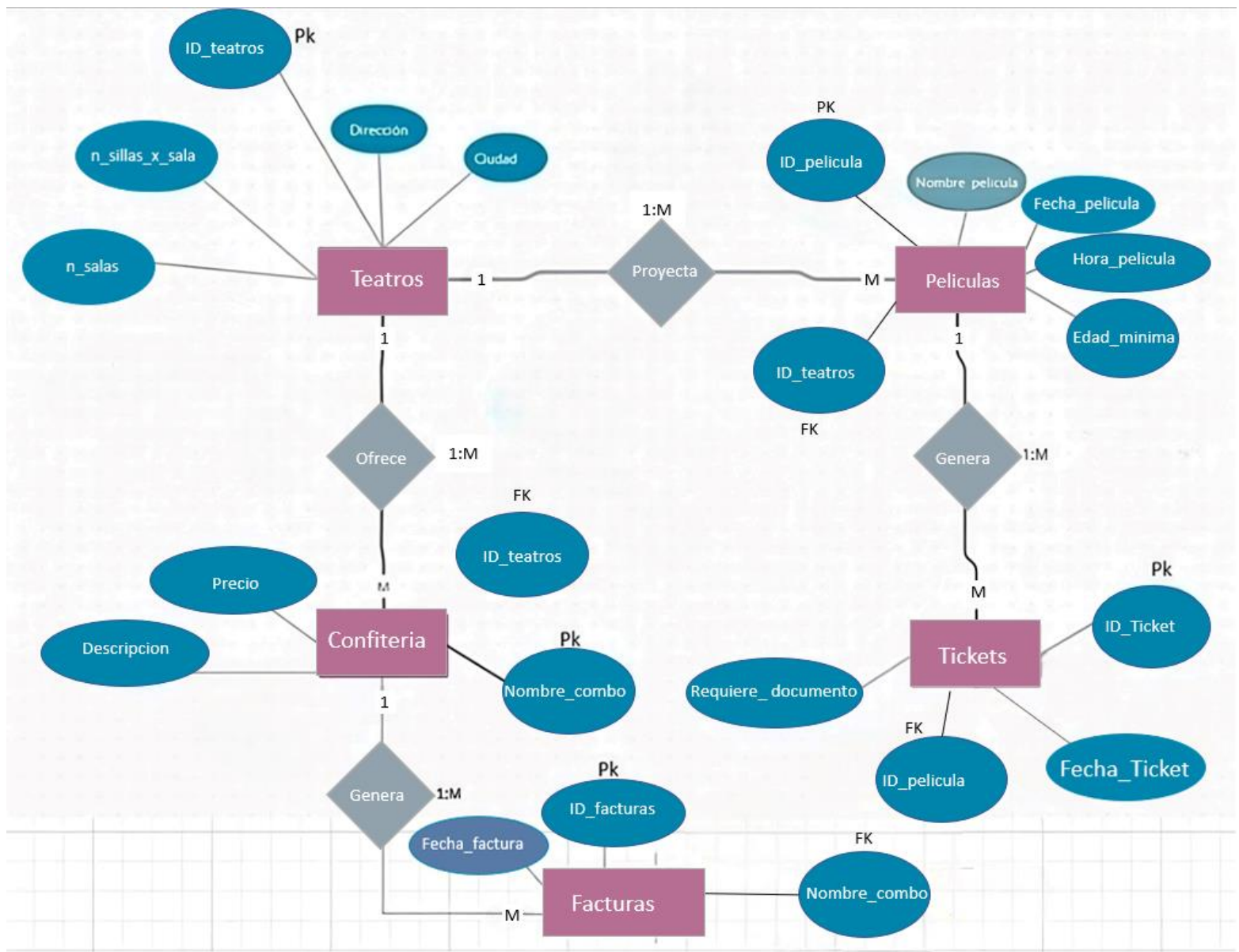
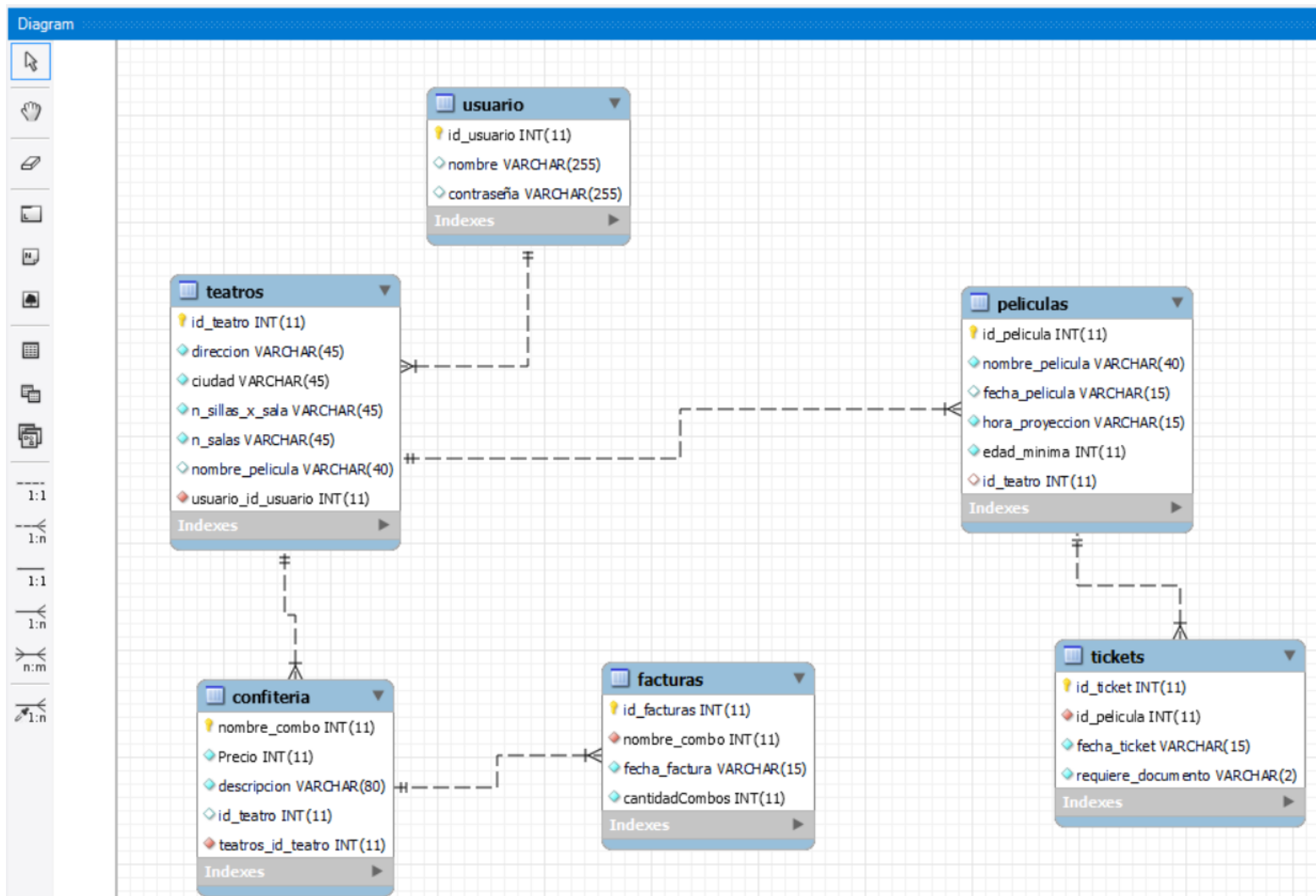


Diagrama MySQL Workbench:



Clases para la conexión a la base de datos:

Esta clase (Conexion) proporciona métodos para establecer y cerrar la conexión a una base de datos MySQL, lo que facilita la gestión de conexiones en una aplicación Java.

```

1  package Basedatos;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  public class Conexion {
8      // Parámetros de conexión
9      private static final String URL = "jdbc:mysql://localhost:3306/cinecolombia";
10     private static final String USUARIO = "root"; // Cambia esto por tu usuario de MySQL
11     private static final String CONTRASEÑA = "123456789"; // Cambia esto por tu contraseña de MySQL
12
13     // Método para obtener una conexión a la base de datos
14     public static Connection obtenerConexion() throws SQLException {
15         Connection conexion = null;
16         try {
17             // Registrar el controlador JDBC
18             Class.forName(className: "com.mysql.cj.jdbc.Driver");
19
20             // Establecer la conexión
21             conexion = DriverManager.getConnection(url:URL, user:USUARIO, password:CONTRASEÑA);
22             System.out.println("¡Conexión exitosa!");
23         } catch (ClassNotFoundException | SQLException e) {
24             System.out.println("Error al conectar a la base de datos: " + e.getMessage());
25             throw new SQLException("Error al conectar a la base de datos: " + e.getMessage());
26         }
27         return conexion;
28     }
29
30     // Método para cerrar la conexión
31     public static void cerrarConexion(Connection conexion) {
32         if (conexion != null) {
33             try {
34                 conexion.close();
35                 System.out.println("Conexión cerrada correctamente.");
36             } catch (SQLException e) {
37                 System.out.println("Error al cerrar la conexión: " + e.getMessage());
38             }
39         }
40     }
41 }
42 
```

1. Primero definimos las variables estáticas URL, USUARIO, y CONTRASEÑA que contienen la información necesaria para establecer la conexión con la base de datos.
2. Método obtenerConexion(): Este método estático devuelve un objeto de tipo Connection que representa la conexión a la base de datos. Dentro del método:
 - Se registra el controlador JDBC específico para MySQL.

- Se establece la conexión utilizando la clase DriverManager, pasando como parámetros la URL de la base de datos, el usuario y la contraseña.
- Si la conexión es exitosa, imprime "¡Conexión exitosa!".

En caso de error, captura la excepción y la imprime en la consola.

3. Método cerrarConexion (): Este método estático se encarga de cerrar la conexión a la base de datos. Toma como parámetro el objeto Connection que se desea cerrar. Dentro del método:

- Verifica que la conexión no sea nula.
- En caso de que la conexión no sea nula, la cierra utilizando el método close ().
- Si se cierra correctamente, imprime "Conexión cerrada correctamente".
- En caso de error al cerrar la conexión, captura la excepción y la imprime en la consola.

.

Este código en Java define una clase llamada Consultas que contiene métodos para realizar consultas a una base de datos.

```

1 package Basedatos;
2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7
8 public class Consultas {
9
10     // Método para validar el nombre de usuario y contraseña
11     public static boolean validarUsuario(String nombre, String contraseña) {
12         Connection conexion = null;
13         PreparedStatement statement = null;
14         ResultSet resultSet = null;
15
16         try {
17             // Obtener la conexión
18             conexion = Conexion.obtenerConexion();
19
20             // Crear la declaración preparada
21             String consulta = "SELECT * FROM cinecolombia.usuario WHERE nombre = ? AND contraseña = ?";
22             statement = conexion.prepareStatement(consulta);
23             statement.setString(1, nombre);
24             statement.setString(2, contraseña);
25
26             // Ejecutar la consulta
27             resultSet = statement.executeQuery();
28
29             // Si hay resultados, el usuario y la contraseña son correctos
30             return resultSet.next();
31
32         } catch (SQLException e) {
33             System.out.println("Error al validar el usuario: " + e.getMessage());
34             return false;
35         } finally {
36             // Cerrar recursos
37             if (resultSet != null) {
38                 try {
39                     resultSet.close();
40                 } catch (SQLException e) {
41                     System.out.println("Error al cerrar el ResultSet: " + e.getMessage());
42                 }
43             }
44             if (statement != null) {
45                 try {
46                     statement.close();
47                 } catch (SQLException e) {
48                     System.out.println("Error al cerrar el Statement: " + e.getMessage());
49                 }
50             }
51             if (conexion != null) {
52                 Conexion.cerrarConexion(conexion);
53             }
54         }
55     }
56
57     public static void consultarTeatroPorIdCine(int id_teatro) {
58         Connection conexion = null;
59         PreparedStatement statement = null;
60         ResultSet resultSet = null;
61
62         try {

```

```

63 // Obtener la conexión
64 conexion = Conexion.obtenerConexion();
65
66 // Crear la consulta
67 String consulta = "SELECT * FROM teatros WHERE id_cine = ?";
68 statement = conexion.prepareStatement(string: consulta);
69 statement.setInt(i: 1, i1: id_teatro);
70
71 // Ejecutar la consulta
72 resultSet = statement.executeQuery();
73
74 // Procesar los resultados
75 while (resultSet.next()) {
76     int idCineResultado = resultSet.getInt(string: "id_teatro");
77     String direccion = resultSet.getString(string: "direccion");
78     String ciudad = resultSet.getString(string: "ciudad");
79     int nSillasXSala = resultSet.getInt(string: "n_sillas_x_sala");
80     int nSalas = resultSet.getInt(string: "n_salas");
81 }
82 } catch (SQLException e) {
83     System.out.println("Error al consultar la tabla teatros: " + e.getMessage());
84 } finally {
85     // Cerrar recursos
86     if (resultSet != null) {
87         try {
88             resultSet.close();
89         } catch (SQLException e) {
90             System.out.println("Error al cerrar el ResultSet: " + e.getMessage());
91         }
92     }
93     if (statement != null) {
94         try {
95             statement.close();
96         } catch (SQLException e) {
97             System.out.println("Error al cerrar el Statement: " + e.getMessage());
98         }
99     }
100     if (conexion != null) {
101         Conexion.cerrarConexion(conexion);
102     }
103 }
104 }
105
106 public static ResultSet consultarConfiteriaPorNombreCombo(String nombreCombo) {
107     Connection conexion = null;
108     PreparedStatement statement = null;
109     ResultSet resultSet = null;
110
111     try {
112         // Obtener la conexión
113         conexion = Conexion.obtenerConexion();
114
115         // Crear la consulta
116         String consulta = "SELECT * FROM confiteria WHERE nombre_combo = ?";
117         statement = conexion.prepareStatement(string: consulta);
118         statement.setString(i: 1, string: nombreCombo);
119
120         // Ejecutar la consulta
121         resultSet = statement.executeQuery();
122
123         // No es necesario procesar los resultados aquí
124     }

```

```

125     } catch (SQLException e) {
126         System.out.println("Error al consultar la tabla confiteria: " + e.getMessage());
127     } finally {
128         // No es necesario cerrar los recursos aquí
129     }
130
131     // Retornar el ResultSet para que sea procesado fuera del método
132     return resultSet;
133 }
134
135 // Método para consultar la tabla peliculas por nombre de película
136 public static ResultSet consultarPeliculaPorId(int idPelicula) {
137     Connection conexion = null;
138     PreparedStatement statement = null;
139     ResultSet resultSet = null;
140
141     try {
142         // Obtener la conexión
143         conexion = Conexion.obtenerConexion();
144
145         // Crear la consulta
146         String consulta = "SELECT * FROM peliculas WHERE id_pelicula = ?";
147         statement = conexion.prepareStatement(statement);
148         statement.setInt(1, idPelicula);
149
150         // Ejecutar la consulta
151         resultSet = statement.executeQuery();
152
153         // No es necesario procesar los resultados aquí
154
155     } catch (SQLException e) {
156
157         System.out.println("Error al consultar la tabla peliculas: " + e.getMessage());
158     } finally {
159         // No es necesario cerrar los recursos aquí
160     }
161
162     // Retornar el ResultSet para que sea procesado fuera del método
163     return resultSet;
164 }
165
166 public static ResultSet mostrarTodasLasPeliculas() {
167     Connection conexion = null;
168     PreparedStatement statement = null;
169     ResultSet resultSet = null;
170
171     try {
172         // Obtener la conexión
173         conexion = Conexion.obtenerConexion();
174
175         // Crear la consulta
176         String consulta = "SELECT * FROM peliculas";
177         statement = conexion.prepareStatement(statement);
178
179         // Ejecutar la consulta
180         resultSet = statement.executeQuery();
181
182         // Retornar el ResultSet
183         return resultSet;
184     } catch (SQLException e) {
185         System.out.println("Error al consultar la tabla de peliculas: " + e.getMessage());

```

```

186         // En caso de error, retornar null
187         return null;
188     } finally {
189         // No es necesario cerrar los recursos aquí
190     }
191 }
192
193 // Método para mostrar toda la tabla de teatros
194 public static ResultSet mostrarTodosLosTeatros() {
195     Connection conexion = null;
196     PreparedStatement statement = null;
197     ResultSet resultSet = null;
198
199     try {
200         // Obtener la conexión
201         conexion = Conexion.obtenerConexion();
202
203         // Crear la consulta
204         String consulta = "SELECT * FROM teatros";
205         statement = conexion.prepareStatement(consulta);
206
207         // Ejecutar la consulta
208         resultSet = statement.executeQuery();
209
210         // No es necesario procesar los resultados aquí
211
212     } catch (SQLException e) {
213         System.out.println("Error al consultar la tabla de teatros: " + e.getMessage());
214     } finally {
215         // No es necesario cerrar los recursos aquí
216     }
217
218     // Retornar el ResultSet para que sea procesado fuera del método
219     return resultSet;
220 }
221
222 // Método para mostrar toda la tabla de confitería
223 public static ResultSet mostrarTodaLaConfiteria() {
224     Connection conexion = null;
225     PreparedStatement statement = null;
226     ResultSet resultSet = null;
227
228     try {
229         // Obtener la conexión
230         conexion = Conexion.obtenerConexion();
231
232         // Crear la consulta
233         String consulta = "SELECT * FROM confiteria";
234         statement = conexion.prepareStatement(consulta);
235
236         // Ejecutar la consulta
237         resultSet = statement.executeQuery();
238
239         // No es necesario procesar los resultados aquí
240
241     } catch (SQLException e) {
242         System.out.println("Error al consultar la tabla de confitería: " + e.getMessage());
243         // En caso de error, retornar null o un ResultSet vacío según lo que necesites en tu lógica
244     } finally {
245         // No cerramos los recursos aquí para permitir que el ResultSet esté disponible para su procesamiento
246         // Los recursos se cerrarán en el método que llama a mostrarTodaLaConfiteria()
247     }

```

```

248
249 // Retornar el ResultSet para que sea procesado fuera del método
250 return resultSet;
251 }
252
253 // Método para consultar una factura por su ID
254 public static void consultarFacturaPorId(int idFactura) {
255     Connection conexion = null;
256     PreparedStatement statement = null;
257     ResultSet resultSet = null;
258
259     try {
260         // Obtener la conexión
261         conexion = Conexion.obtenerConexion();
262
263         // Crear la consulta
264         String consulta = "SELECT * FROM facturas WHERE id_facturas = ?";
265         statement = conexion.prepareStatement(consulta);
266         statement.setInt(1, idFactura);
267
268         // Ejecutar la consulta
269         resultSet = statement.executeQuery();
270
271         // Procesar y mostrar los resultados
272         if (resultSet.next()) {
273             String nombreCombo = resultSet.getString("nombre_combo");
274             String fechaFactura = resultSet.getString("fecha_factura");
275
276             System.out.println("ID Factura: " + idFactura + ", Nombre Combo: " + nombreCombo + ", Fecha: " + fechaFactura);
277         } else {
278             System.out.println("No se encontró una factura con el ID especificado.");
279         }
280     } catch (SQLException e) {
281         System.out.println("Error al consultar la factura: " + e.getMessage());
282     } finally {
283         // Cerrar recursos
284         if (resultSet != null) {
285             try {
286                 resultSet.close();
287             } catch (SQLException e) {
288                 System.out.println("Error al cerrar el ResultSet: " + e.getMessage());
289             }
290         }
291         if (statement != null) {
292             try {
293                 statement.close();
294             } catch (SQLException e) {
295                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
296             }
297         }
298         if (conexion != null) {
299             Conexion.cerrarConexion(conexion);
300         }
301     }
302 }
303
304 // Método para consultar un ticket por su ID
305 public static void consultarTicketPorId(int idTicket) {
306     Connection conexion = null;
307     PreparedStatement statement = null;
308     ResultSet resultSet = null;
309

```

```

310
311     try {
312         // Obtener la conexión
313         conexion = Conexion.obtenerConexion();
314
315         // Crear la consulta
316         String consulta = "SELECT * FROM tickets WHERE id_ticket = ?";
317         statement = conexion.prepareStatement(consulta);
318         statement.setInt(1, idTicket);
319
320         // Ejecutar la consulta
321         resultSet = statement.executeQuery();
322
323         // Procesar y mostrar los resultados
324         if (resultSet.next()) {
325             int idPelicula = resultSet.getInt("id_pelicula");
326             String fechaTicket = resultSet.getString("fecha_ticket");
327             boolean requiereDocumento = resultSet.getBoolean("requiere_documento");
328
329             System.out.println("ID Ticket: " + idTicket + ", ID Pelicula: " + idPelicula + ", Fecha: " + fechaTicket + ", Requiere Documento: " + requiereDocumento);
330         } else {
331             System.out.println("No se encontró un ticket con el ID especificado.");
332         }
333     } catch (SQLException e) {
334         System.out.println("Error al consultar el ticket: " + e.getMessage());
335     } finally {
336         // Cerrar recursos
337         if (resultSet != null) {
338             try {
339                 resultSet.close();
340             } catch (SQLException e) {
341                 System.out.println("Error al cerrar el ResultSet: " + e.getMessage());
342             }
343         }
344         if (statement != null) {
345             try {
346                 statement.close();
347             } catch (SQLException e) {
348                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
349             }
350         }
351         if (conexion != null) {
352             Conexion.cerrarConexion(conexion);
353         }
354     }
355
356     public static ResultSet consultarUltimoIdTicket() throws SQLException {
357         Connection conexion = null;
358         PreparedStatement statement = null;
359         ResultSet resultSet = null;
360
361         try {
362             // Obtener la conexión
363             conexion = Conexion.obtenerConexion();
364
365             // Crear la consulta para obtener el último ID de ticket
366             String consulta = "SELECT MAX(id_ticket) FROM tickets";
367             statement = conexion.prepareStatement(consulta);
368
369             // Ejecutar la consulta
370             resultSet = statement.executeQuery();
371
372             return resultSet;
373         } finally {
374             // No cerramos recursos aquí para permitir que el llamador maneje el cierre de la conexión y el statement
375         }
376     }
377

```

1. `validarUsuario (String nombre, String contraseña)`: Este método valida un usuario y una contraseña en la base de datos. Realiza una consulta SQL para verificar si existe un usuario con el nombre y la contraseña proporcionados. Retorna `true` si la consulta devuelve resultados, indicando que el usuario y la contraseña son válidos.
2. `consultarTeatroPorIdCine (int id_teatro)`: Este método consulta teatros por su ID de cine. Realiza una consulta SQL para obtener información sobre los teatros relacionados con un cine específico, utilizando el ID del cine como parámetro de consulta.
3. `consultarConfiteriaPorNombreCombo (String nombreCombo)`: Este método consulta la confitería por el nombre del combo. Realiza una consulta SQL para obtener información sobre los combos de la confitería que coincidan con el nombre proporcionado como parámetro de consulta. Retorna un `ResultSet` que contiene los resultados de la consulta.
4. `consultarPeliculaPorId (int idPelicula)`: Este método consulta una película por su ID. Realiza una consulta SQL para obtener información sobre una película específica utilizando su ID como parámetro de consulta. Retorna un `ResultSet` que contiene los resultados de la consulta.
5. `mostrarTodasLasPeliculas ()`: Este método consulta todas las películas en la base de datos. Realiza una consulta SQL para obtener información sobre todas las películas almacenadas en la tabla películas. Retorna un `ResultSet` que contiene los resultados de la consulta.
6. `mostrarTodosLosTeatros ()`: Este método consulta todos los teatros en la base de datos. Realiza una consulta SQL para obtener información sobre todos los teatros almacenados en la tabla teatros. Retorna un `ResultSet` que contiene los resultados de la consulta.
7. `mostrarTodaLaConfiteria ()`: Este método consulta toda la confitería en la base de datos. Realiza una consulta SQL para obtener información sobre todos los productos de la confitería almacenados en la tabla confiteria. Retorna un `ResultSet` que contiene los resultados de la consulta.
8. `consultarFacturaPorId (int idFactura)`: Este método consulta una factura por su ID. Realiza una consulta SQL para obtener información sobre una factura específica utilizando su ID como parámetro de consulta. Muestra los resultados por consola.
9. `consultarTicketPorId (int idTicket)`: Este método consulta un ticket por su ID. Realiza una consulta SQL para obtener información sobre un ticket específico utilizando su ID como parámetro de consulta. Muestra los resultados por consola.
10. `consultarUltimoIdTicket ()`: Este método consulta el último ID de ticket. Realiza una consulta SQL para obtener el máximo ID de ticket en la tabla tickets. Retorna un `ResultSet` que contiene el resultado de la consulta.

define una clase llamada “Introducir y eliminar”, que contiene métodos para insertar y eliminar registros en diferentes tablas de la base de datos.

```

1 package Basedatos;
2 import java.sql.Connection;
3 import java.sql.PreparedStatement;
4 import java.sql.SQLException;
5 import java.sql.ResultSet;
6
7 public class Introducir y eliminar {
8
9     // Método para insertar una nueva película
10    public static void insertarPelícula(String nombrePelícula, String fechaPelícula, String horaProyeccion, int edadMinima
11        Connection conexion = null;
12        PreparedStatement statement = null;
13
14    try {
15        // Obtener la conexión
16        conexion = Conexion.obtenerConexion();
17
18        // Crear la consulta de inserción
19        String consulta = "INSERT INTO peliculas (nombre_pelicula, fecha_pelicula, hora_proyeccion, edad_minima) VALUES (?, ?, ?, ?)";
20        statement = conexion.prepareStatement(consulta);
21
22        // Establecer los parámetros
23        statement.setString(1, nombrePelícula);
24        statement.setString(2, fechaPelícula);
25        statement.setString(3, horaProyeccion);
26        statement.setInt(4, edadMinima);
27
28        // Ejecutar la consulta de inserción
29        statement.executeUpdate();
30
31        System.out.println("¡Película insertada correctamente!");
32    } catch (SQLException e) {
33        System.out.println("Error al insertar película: " + e.getMessage());
34    } finally {
35        // Cerrar recursos
36        if (statement != null) {
37            try {
38                statement.close();
39            } catch (SQLException e) {
40                System.out.println("Error al cerrar el Statement: " + e.getMessage());
41            }
42        }
43        if (conexion != null) {
44            Conexion.cerrarConexion(conexion);
45        }
46    }
47 }
48
49 // Método para insertar un nuevo teatro
50 public static void insertarTeatro(int id_teatro, String direccion, String ciudad, int n_sillas_x_sala, int n_salas) {
51     Connection conexion = null;
52     PreparedStatement statement = null;
53
54     try {
55         // Obtener la conexión
56         conexion = Conexion.obtenerConexion();
57
58         // Crear la consulta de inserción
59         String consulta = "INSERT INTO teatros (id_teatro, direccion, ciudad, n_sillas_x_sala, n_salas) VALUES (?, ?, ?, ?, ?)";
60         statement = conexion.prepareStatement(consulta);
61
62         // Establecer los parámetros

```

```

63         statement.setInt(i: 1, i1: id_teatro);
64         statement.setString(i: 2, string: direccion);
65         statement.setString(i: 3, string: ciudad);
66         statement.setInt(i: 4, i1: n_sillas_x_sala);
67         statement.setInt(i: 5, i1: n_salas);
68
69         // Ejecutar la consulta de inserción
70         statement.executeUpdate();
71
72         System.out.println("¡Teatro insertado correctamente!");
73     } catch (SQLException e) {
74         System.out.println("Error al insertar teatro: " + e.getMessage());
75     } finally {
76         // Cerrar recursos
77         if (statement != null) {
78             try {
79                 statement.close();
80             } catch (SQLException e) {
81                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
82             }
83         }
84         if (conexion != null) {
85             Conexion.cerrarConexion(conexion);
86         }
87     }
88 }
89
90 // Método para insertar un nuevo combo en la confitería
91 public static void insertarCombo(String nombreCombo, int precio, String descripcion) {
92     Connection conexion = null;
93     PreparedStatement statement = null;
94
95     try {
96         // Obtener la conexión
97         conexion = Conexion.obtenerConexion();
98
99         // Crear la consulta de inserción
100        String consulta = "INSERT INTO confiteria (nombre_combo, precio, descripcion) VALUES (?, ?, ?)";
101        statement = conexion.prepareStatement(string: consulta);
102
103        // Establecer los parámetros
104        statement.setString(i: 1, string: nombreCombo);
105        statement.setDouble(i: 2, d: precio);
106        statement.setString(i: 3, string: descripcion);
107
108        // Ejecutar la consulta de inserción
109        statement.executeUpdate();
110
111        System.out.println("¡Combo insertado correctamente!");
112    } catch (SQLException e) {
113        System.out.println("Error al insertar combo: " + e.getMessage());
114    } finally {
115        // Cerrar recursos
116        if (statement != null) {
117            try {
118                statement.close();
119            } catch (SQLException e) {
120                System.out.println("Error al cerrar el Statement: " + e.getMessage());
121            }
122        }
123        if (conexion != null) {
124            Conexion.cerrarConexion(conexion);

```

```

125     }
126   }
127 }
128
129 // Método para eliminar una película por su ID
130 public static void eliminarPelículaPorId(int idPelícula) {
131     Connection conexion = null;
132     PreparedStatement statement = null;
133
134     try {
135         // Obtener la conexión
136         conexion = Conexion.obtenerConexion();
137
138         // Crear la consulta de eliminación
139         String consulta = "DELETE FROM peliculas WHERE id_pelicula = ?";
140         statement = conexion.prepareStatement(consulta);
141
142         // Establecer el parámetro
143         statement.setInt(1, idPelícula);
144
145         // Ejecutar la consulta de eliminación
146         statement.executeUpdate();
147
148         System.out.println("Película eliminada correctamente!");
149     } catch (SQLException e) {
150         System.out.println("Error al eliminar película: " + e.getMessage());
151     } finally {
152         // Cerrar recursos
153         if (statement != null) {
154             try {
155                 statement.close();

```

```

156             } catch (SQLException e) {
157                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
158             }
159         }
160         if (conexion != null) {
161             Conexion.cerrarConexion(conexion);
162         }
163     }
164 }
165
166 // Método para insertar un nuevo ticket
167 public static void insertarTicket(int idPelícula, String fechaTicket, String requiereDocumento) {
168     Connection conexion = null;
169     PreparedStatement statement = null;
170
171     try {
172         // Obtener la conexión
173         conexion = Conexion.obtenerConexion();
174
175         // Crear la consulta de inserción
176         String consulta = "INSERT INTO tickets (id_pelicula, fecha_ticket, requiere_documento) VALUES (?, ?, ?)";
177         statement = conexion.prepareStatement(consulta);
178
179         // Establecer los parámetros
180         statement.setInt(1, idPelícula);
181         statement.setString(2, fechaTicket);
182         statement.setString(3, requiereDocumento);
183
184         // Ejecutar la consulta de inserción
185         statement.executeUpdate();
186
187         System.out.println("Ticket insertado correctamente!");

```

```
187     } catch (SQLException e) {
188         System.out.println("Error al insertar ticket: " + e.getMessage());
189     } finally {
190         // Cerrar recursos
191         if (statement != null) {
192             try {
193                 statement.close();
194             } catch (SQLException e) {
195                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
196             }
197         }
198         if (conexion != null) {
199             Conexion.cerrarConexion(conexion);
200         }
201     }
202 }
203
204 // Método para insertar una nueva factura
205 public static int insertarFactura(String nombre, String fecha) {
206     Connection conexion = null;
207     PreparedStatement statement = null;
208     ResultSet resultSet = null;
209     int id_facturas = 0;
210
211     try {
212         conexion = Conexion.obtenerConexion();
213         String consulta = "INSERT INTO facturas (nombre_combo, fecha_factura) VALUES (?, ?)";
214         statement = conexion.prepareStatement(string: consulta, new String[]{"id_facturas"});
215         statement.setString(i: 1, string: nombre);
216         statement.setString(i: 2, string: fecha);
217         statement.executeUpdate();
```

```

218
219         resultSet = statement.getGeneratedKeys();
220         if (resultSet.next()) {
221             id_facturas = resultSet.getInt(1);
222         }
223     } catch (SQLException e) {
224         System.out.println("Error al insertar la factura: " + e.getMessage());
225     } finally {
226         // Cerrar recursos
227         if (resultSet != null) {
228             try {
229                 resultSet.close();
230             } catch (SQLException e) {
231                 System.out.println("Error al cerrar el ResultSet: " + e.getMessage());
232             }
233         }
234         if (statement != null) {
235             try {
236                 statement.close();
237             } catch (SQLException e) {
238                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
239             }
240         }
241         if (conexion != null) {
242             Conexion.cerrarConexion(conexion);
243         }
244     }
245
246     return id_facturas;
247 }
248
249 // Método para eliminar un ticket por su ID
250 public static void eliminarTicketPorId(int idTicket) {
251     Connection conexion = null;
252     PreparedStatement statement = null;
253
254     try {
255         // Obtener la conexión
256         conexion = Conexion.obtenerConexion();
257
258         // Crear la consulta de eliminación
259         String consulta = "DELETE FROM tickets WHERE id_ticket = ?";
260         statement = conexion.prepareStatement(consulta);
261
262         // Establecer el parámetro
263         statement.setInt(1, idTicket);
264
265         // Ejecutar la consulta de eliminación
266         statement.executeUpdate();
267
268         System.out.println("Ticket eliminado correctamente!");
269     } catch (SQLException e) {
270         System.out.println("Error al eliminar ticket: " + e.getMessage());
271     } finally {
272         // Cerrar recursos
273         if (statement != null) {
274             try {
275                 statement.close();
276             } catch (SQLException e) {
277                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
278             }
279         }

```

```

280 |         if (conexion != null) {
281 |             Conexion.cerrarConexion(conexion);
282 |         }
283 |     }
284 | }
285 |
286 | // Método para eliminar una factura por su ID
287 | public static void eliminarFacturaPorId(int idFactura) {
288 |     Connection conexion = null;
289 |     PreparedStatement statement = null;
290 |
291 |     try {
292 |         // Obtener la conexión
293 |         conexion = Conexion.obtenerConexion();
294 |
295 |         // Crear la consulta de eliminación
296 |         String consulta = "DELETE FROM facturas WHERE id_facturas = ?";
297 |         statement = conexion.prepareStatement(string: consulta);
298 |
299 |         // Establecer el parámetro
300 |         statement.setInt(1, idFactura);
301 |
302 |         // Ejecutar la consulta de eliminación
303 |         statement.executeUpdate();
304 |
305 |         System.out.println("Factura eliminada correctamente!");
306 |     } catch (SQLException e) {
307 |         System.out.println("Error al eliminar factura: " + e.getMessage());
308 |     } finally {
309 |         // Cerrar recursos
310 |         if (statement != null) {
311 |             try {
312 |                 statement.close();
313 |             } catch (SQLException e) {
314 |                 System.out.println("Error al cerrar el Statement: " + e.getMessage());
315 |             }
316 |         }
317 |         if (conexion != null) {
318 |             Conexion.cerrarConexion(conexion);
319 |         }
320 |     }
321 | }
322 |

```

1. insertarPelicula (String nombrePelicula, String fechaPelicula, String horaProyeccion, int edadMinima): Inserta una nueva película en la base de datos. Toma como parámetros el nombre de la película, la fecha de proyección, la hora de proyección y la edad mínima requerida para verla.

2. insertarTeatro (int id_teatro, String direccion, String ciudad, int n_sillas_x_sala, int n_salas): Inserta un nuevo teatro en la base de datos. Toma como parámetros el ID del teatro, la dirección, la ciudad, el número de sillas por sala y el número de salas.
3. insertarCombo (String nombreCombo, int precio, String descripcion): Inserta un nuevo combo en la confitería. Toma como parámetros el nombre del combo, el precio y una descripción.
4. eliminarPeliculaPorId (int idPelicula): Elimina una película de la base de datos por su ID.
5. insertarTicket (int idPelicula, String fechaTicket, String requiereDocumento): Inserta un nuevo ticket en la base de datos. Toma como parámetros el ID de la película, la fecha del ticket y un indicador de si se requiere documento para la entrada.
6. insertarFactura (String nombre, String fecha): Inserta una nueva factura en la base de datos y retorna el ID de la factura insertada.
7. eliminarTicketPorId (int idTicket): Elimina un ticket de la base de datos por su ID.
8. eliminarFacturaPorId (int idFactura): Elimina una factura de la base de datos por su ID.

Cada método realiza las operaciones de inserción o eliminación correspondientes en la base de datos, maneja las excepciones de SQL y cierra adecuadamente las conexiones y recursos asociados.

Clases para la ejecución:

Se define una clase llamada “PeliculasManager” en el paquete Ejecutables

```
1 package Ejecutables;
2
3 import Basedatos.Consultas;
4 import Basedatos.IntroducirYeliminar;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7
8 public class PeliculasManager {
9     public PeliculasManager() {
10     }
11
12
13     public void agregarPelicula(String nombre, String fecha, String horaProyeccion, int edadMinima) {
14         IntroducirYeliminar.insertarPelicula(nombrePelicula: nombre, fechaPelicula: fecha, horaProyeccion, edadMinima);
15         System.out.println("Pelicula agregada con éxito.");
16     }
17
18     public void mostrarPeliculasDisponibles() {
19         ResultSet resultSet = Consultas.mostrarTodasLasPeliculas();
20         try {
21             System.out.println("Peliculas Disponibles:");
22             while (resultSet.next()) {
23                 int idPelicula = resultSet.getInt(string: "id_pelicula");
24                 String nombrePelicula = resultSet.getString(string: "nombre_pelicula");
25                 String fechaPelicula = resultSet.getString(string: "fecha_pelicula");
26                 String horaProyeccion = resultSet.getString(string: "hora_proyeccion");
27                 int edadMinima = resultSet.getInt(string: "edad_minima");
28
29                 System.out.println("ID de pelicula: " + idPelicula +
30                     ", Nombre: " + nombrePelicula +
31                     ", Fecha: " + fechaPelicula +
32                     ", Hora: " + horaProyeccion);
33             }
34         } catch (SQLException e) {
35             e.printStackTrace();
36         }
37     }
38 }
```

```
33 |         }  
34 |     } catch (SQLException e) {  
35 |         System.out.println("Error al mostrar las películas: " + e.getMessage());  
36 |     }  
37 |  
38 |  
39 |     public void eliminarPelicula(int idPelicula) {  
40 |         IntroducirYeliminar.eliminarPeliculaPorId(idPelicula);  
41 |         System.out.println("Película eliminada con éxito.");  
42 |     }  
43 |  
44 | }
```

1. `agregarPelicula (String nombre, String fecha, String horaProyeccion, int edadMinima)`: Este método permite agregar una nueva película a la base de datos. Toma como parámetros el nombre de la película, la fecha de proyección, la hora de proyección y la edad mínima requerida para verla. Utiliza el método `insertarPelicula` de la clase `IntroducirYeliminar` para realizar la inserción en la base de datos.
2. `mostrarPeliculasDisponibles ()`: Este método muestra todas las películas disponibles en la base de datos. Utiliza el método `mostrarTodasLasPeliculas` de la clase `Consultas` para obtener un `ResultSet` que contiene la información de las películas. Luego, itera sobre este conjunto de resultados y muestra los detalles de cada película por consola.
3. `eliminarPelicula (int idPelicula)`: Este método permite eliminar una película de la base de datos por su ID. Utiliza el método `eliminarPeliculaPorId` de la clase `IntroducirYeliminar` para realizar la eliminación en la base de datos.

Cada método realiza su operación correspondiente en la base de datos y maneja las excepciones de SQL adecuadamente. Además, imprime mensajes en consola para informar sobre el resultado de las operaciones realizadas.

Se define una clase llamada “TeatrosManager” en el paquete Ejecutables

```

1  package Ejecutables;
2
3  import Basedatos.Consultas;
4  import Basedatos.Introduciryeliminar;
5  import javax.swing.*;
6  import java.awt.*;
7  import java.sql.ResultSet;
8  import java.sql.SQLException;
9
10 public class TeatrosManager {
11     public void agregarTeatro(int id_teatro, String direccion, String ciudad, int n_sillas_x_sala, int n_salas) {
12         Introduciryeliminar.insertarTeatro(id_teatro, direccion, ciudad, n_sillas_x_sala, n_salas);
13         System.out.println("Teatro agregado con éxito. ID de teatro: " + id_teatro);
14     }
15
16     public void mostrarTeatrosDisponibles() {
17         StringBuilder teatrosInfo = new StringBuilder();
18         teatrosInfo.append("Teatros Disponibles:\n");
19         ResultSet resultSet = Consultas.mostrarTodosLosTeatros();
20         try {
21             while (resultSet.next()) {
22                 String ID_teatro = resultSet.getString("id_cine");
23                 String direccion = resultSet.getString("direccion");
24                 String ciudad = resultSet.getString("ciudad");
25                 int n_sillas_x_sala = resultSet.getInt("n_sillas_x_sala");
26                 int n_salas = resultSet.getInt("n_salas");
27
28                 teatrosInfo.append("ID de teatro: ").append(ID_teatro).append("\n");
29                 teatrosInfo.append("Dirección: ").append(direccion).append("\n");
30                 teatrosInfo.append("Ciudad: ").append(ciudad).append("\n");
31                 teatrosInfo.append("Número de sillas por sala: ").append(n_sillas_x_sala).append("\n");
32                 teatrosInfo.append("Salas: ").append(n_salas).append("\n\n");
33             }
34             mostrarEnInterfazGrafica(info: teatrosInfo.toString());
35         } catch (SQLException e) {
36             System.out.println("Error al mostrar los teatros disponibles: " + e.getMessage());
37         }
38     }
39
40     private void mostrarEnInterfazGrafica(String info) {
41         JFrame frame = new JFrame(title: "Teatros Disponibles");
42         frame.setDefaultCloseOperation(operation: JFrame.DISPOSE_ON_CLOSE);
43
44         JPanel panel = new JPanel(new BorderLayout());
45         JTextArea textArea = new JTextArea(rows: 10, columns: 40);
46         textArea.setEditable(b: false);
47         JScrollPane scrollPane = new JScrollPane(view: textArea);
48         textArea.setText(s: info);
49
50         panel.add(comp: scrollPane, constraints: BorderLayout.CENTER);
51         frame.getContentPane().add(comp: panel);
52         frame.pack();
53         frame.setVisible(b: true);
54     }
55 }

```

1. agregarTeatro (int id_teatro, String direccion, String ciudad, int n_sillas_x_sala, int n_salas): Este método permite agregar un nuevo teatro a la base de datos. Toma como parámetros el ID del teatro, la dirección, la ciudad, el número de sillas por

sala y el número de salas. Utiliza el método insertarTeatro de la clase Introducir y eliminar para realizar la inserción en la base de datos.

2. mostrarTeatrosDisponibles (): Este método muestra todos los teatros disponibles en la base de datos. Utiliza el método mostrarTodosLosTeatros de la clase Consultas para obtener un ResultSet que contiene la información de los teatros. Luego, itera sobre este conjunto de resultados y construye un StringBuilder con la información de los teatros. Después, llama al método mostrarEnInterfazGrafica para mostrar la información en una interfaz gráfica.
3. mostrarEnInterfazGrafica (String info): Este método muestra la información proporcionada en una interfaz gráfica utilizando Swing. Crea un nuevo JFrame y un JTextArea para mostrar el texto, luego lo coloca en un JPanel con un JScrollPane y lo agrega al JFrame.

Cada método realiza su operación correspondiente en la base de datos y maneja las excepciones de SQL adecuadamente. Además, muestra la información de manera visual utilizando una interfaz gráfica.

Se define una clase llamada (ConfiteriaManager) en el paquete Ejecutables

```
1 package Ejecutables;
2
3 import Basedatos.Consultas;
4 import Basedatos.Introducir y eliminar;
5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7
8 public class ConfiteriaManager {
9
10     public void agregarCombo(String nombre, int precio, String descripcion) {
11         Introducir y eliminar.insertarCombo(nombreCombo:nombre, precio, descripcion);
12         System.out.println("Combo agregado con éxito.");
13     }
14
15     public void mostrarCombosDisponibles() {
16         System.out.println("Combos Disponibles:");
17         ResultSet resultSet = Consultas.mostrarTodaLaConfiteria();
18         try {
19             while (resultSet.next()) {
20                 String nombreCombo = resultSet.getString(string: "nombre_combo");
21                 int precio = resultSet.getInt(string: "precio");
22                 String descripcion = resultSet.getString(string: "descripcion");
23
24                 System.out.println("Nombre: " + nombreCombo +
25                                     ", Precio: " + precio +
26                                     ", Descripción: " + descripcion);
27             }
28         } catch (SQLException e) {
29             System.out.println("Error al mostrar combos disponibles: " + e.getMessage());
30         }
31     }
32 }
```

```

33 public void comprarCombo(String nombreCombo, String fechaFactura) {
34     ResultSet resultSet = Consultas.consultarConfiteriaPorNombreCombo(nombreCombo);
35     try {
36         if (resultSet.next()) {
37             String nombre = resultSet.getString(string: "nombre_combo");
38             int precio = resultSet.getInt(string: "precio");
39             String descripcion = resultSet.getString(string: "descripcion");
40
41             // Insertar la factura
42             IntroducirYeliminar.insertarFactura(nombre, fecha: fechaFactura);
43
44             // Mostrar el detalle de la factura y el combo seleccionado
45             System.out.println("¡Compra realizada con éxito!");
46             System.out.println("----- FACTURA -----");
47             System.out.println("Fecha compra: " + fechaFactura);
48             System.out.println("Combo: " + nombre);
49             System.out.println("Precio: " + precio);
50             System.out.println("Descripción: " + descripcion);
51             System.out.println("-----");
52         } else {
53             System.out.println("El combo seleccionado no está disponible.");
54         }
55     } catch (SQLException e) {
56         System.out.println("Error al comprar combo: " + e.getMessage());
57     }
58 }
59 public ResultSet buscarCombo(String nombreCombo) {
60     return Consultas.consultarConfiteriaPorNombreCombo(nombreCombo);
61 }
62 }

```

1. `agregarCombo (String nombre, int precio, String descripcion)`: Este método permite agregar un nuevo combo a la base de datos. Toma como parámetros el nombre del combo, el precio y la descripción. Utiliza el método `insertarCombo` de la clase `IntroducirYeliminar` para realizar la inserción en la base de datos.
2. `mostrarCombosDisponibles ()`: Este método muestra todos los combos disponibles en la base de datos. Utiliza el método `mostrarTodaLaConfiteria` de la clase `Consultas` para obtener un `ResultSet` que contiene la información de los combos. Luego, itera sobre este conjunto de resultados y muestra la información de cada combo por consola.
3. `comprarCombo (String nombreCombo, String fechaFactura)`: Este método permite comprar un combo seleccionado por su nombre. Utiliza el método `consultarConfiteriaPorNombreCombo` de la clase `Consultas` para verificar si el combo está disponible. Si lo está, obtiene la información del combo y utiliza el método `insertarFactura` de la clase `IntroducirYeliminar` para insertar una nueva factura en la base de datos con la información del combo comprado. Finalmente, muestra los detalles de la factura y del combo comprado por consola.

4. `buscarCombo (String nombreCombo)`: Este método permite buscar un combo por su nombre. Utiliza el método `consultarConfiteriaPorNombreCombo` de la clase `Consultas` para obtener un `ResultSet` que contiene la información del combo buscado, que luego es devuelto para ser procesado fuera del método.

Cada método realiza su operación correspondiente en la base de datos y maneja las excepciones de SQL adecuadamente.

Se define la clase `main` en el que se ejecutara el programa

```
2 package Ejecutables;
3
4 import javax.swing.*;
5
6 public class MainGUI {
7
8     public static void main(String[] args) {
9         SwingUtilities.invokeLater(new Runnable() {
10             public void run() {
11                 new InterfazGUI();
12             }
13         });
14     }
15 }
```

1. `main (String[] args)`: Este método es el punto de entrada de la aplicación. Utiliza `SwingUtilities.invokeLater()` para ejecutar la creación de una nueva instancia de la clase `InterfazGUI` en el hilo de despacho de eventos de Swing. Esto asegura que la creación y manipulación de componentes de la interfaz gráfica se realicen de manera segura en el hilo de eventos de Swing.

4. Resultados

La industria cinematográfica colombiana ha experimentado un crecimiento significativo en los últimos años, impulsada por el aumento de la demanda de entretenimiento y la inversión en nuevas salas de cine. Sin embargo, este crecimiento también ha generado nuevos desafíos, como la necesidad de optimizar la gestión de las operaciones y mejorar la experiencia del usuario.

En este contexto, un equipo de desarrolladores ha creado un prototipo de software para la gestión de salas de cine en Colombia. Este prototipo tiene como objetivo abordar las siguientes problemáticas:

- **Confusión en los turnos:** En algunos cines colombianos, existe una problemática común relacionada con la confusión en los turnos de las salas de cine. Esto genera pérdida de tiempo, errores y una experiencia poco satisfactoria para los usuarios.
- **Falta de automatización:** Los procesos de venta de boletas, compra de alimentos y gestión de salas de cine a menudo se realizan de forma manual, lo que genera lentitud, errores y una experiencia poco fluida para los usuarios.
- **Información desorganizada:** La información sobre las salas de cine, las películas, los turnos, los alimentos y las ventas no siempre está disponible de forma centralizada y organizada, lo que dificulta su acceso y uso por parte del personal y los usuarios.
- **Falta de interactividad:** La interacción entre los usuarios y el sistema de gestión de salas de cine suele ser limitada, lo que reduce la eficiencia y la satisfacción del usuario.

Solución propuesta:

El prototipo de software propuesto busca solucionar las problemáticas mencionadas anteriormente mediante la implementación de las siguientes funcionalidades:

Para usuarios:

- **Acceso a la cartelera:** Los usuarios pueden visualizar la cartelera de películas en curso y próximas, con información sobre horarios, teatros disponibles, combos de comida.
- **Compra de boletas en línea:** El sistema permite a los usuarios comprar boletas para las películas en línea del cine. El sistema también permite generar comprobantes de compra y seleccionar asientos.
- **Compra de alimentos y bebidas:** Los usuarios pueden realizar pedidos de alimentos y bebidas desde la aplicación móvil. El sistema permite visualizar el menú, seleccionar los productos y realizar el pago en línea.

- Visualización de información en tiempo real: Los usuarios pueden acceder a información en tiempo real sobre el estado de las salas de cine, los tiempos de espera para la compra de alimentos y bebidas.
- Historial de compras: Los usuarios pueden consultar el historial de sus compras, tanto de boletas como de alimentos y bebidas.

Para administradores:

- Gestión de salas de cine: Los administradores pueden crear, editar y eliminar salas de cine, asignarles capacidades y configurar los precios de las boletas.
- Gestión de películas: Los administradores pueden agregar, editar y eliminar películas de la cartelera, asignarles horarios y salas de proyección.
- Gestión de alimentos y bebidas: Los administradores pueden crear, editar y eliminar productos del menú, establecer precios y gestionar el inventario.
- Generación de informes y estadísticas: El sistema permite generar informes y estadísticas sobre las ventas de boletas, alimentos y bebidas, la asistencia a las películas y el uso de las salas de cine. Esta información puede ser utilizada para tomar decisiones estratégicas sobre la programación de películas, la gestión de salas, la estrategia de marketing y la oferta de productos.
- Control de acceso: El sistema permite a los administradores crear y gestionar cuentas de usuario para el personal del cine, asignándoles roles y permisos específicos.

Tecnología utilizada:

El prototipo de software ha sido desarrollado utilizando las siguientes tecnologías:

- Lenguaje de programación: Java
- Base de datos: MySQL
- Interfaz de usuario: Gráficas

Beneficios de la solución:

La implementación del prototipo de software propuesto ofrece los siguientes beneficios:

- Mejora en la experiencia del usuario: El sistema ofrece una experiencia de usuario más intuitiva y fluida, lo que permite a los usuarios comprar boletas, acceder a las salas de cine, obtener información sobre las películas y realizar pedidos de alimentos y bebidas de forma rápida y sencilla.
- Mayor eficiencia del personal: El sistema automatiza muchas de las tareas manuales que actualmente realiza el personal del cine, como la venta de boletas, la toma de pedidos de alimentos y bebidas y el control de acceso a las salas de cine. Esto

permite al personal dedicar más tiempo a la atención al cliente y a otras tareas más importantes.

- Reducción de errores y tiempos de espera: El sistema automatiza la venta de boletas, la toma de pedidos de alimentos y bebidas y el control de acceso a las salas de cine, lo que reduce la probabilidad de errores y tiempos de espera.
- Información organizada y de fácil acceso: El sistema almacena y organiza la información sobre las salas de cine, las películas, los turnos, los alimentos, las bebidas y las ventas de forma centralizada en la base de datos MySQL, lo que facilita su acceso y uso por parte del personal y los usuarios.
- Interacción fluida entre el usuario y el sistema: El sistema ofrece una interfaz de usuario intuitiva y fácil de usar, tanto en la web como en la aplicación móvil, lo que permite una interacción fluida y eficiente entre el usuario y el sistema.
- Toma de decisiones estratégicas: La generación de informes y estadísticas permite a los administradores tomar decisiones estratégicas sobre la programación de películas, la gestión de salas, la estrategia de marketing y la oferta de productos.

Menú Inicial:



Inicio de Sesión:

Iniciar sesión



Ingrese su usuario:

Ingrese su contraseña:

OK

Cancelar

Menú Gestionar:

Gestionar

Bienvenido Administrador

Agregar Pelicula

Agregar Teatro

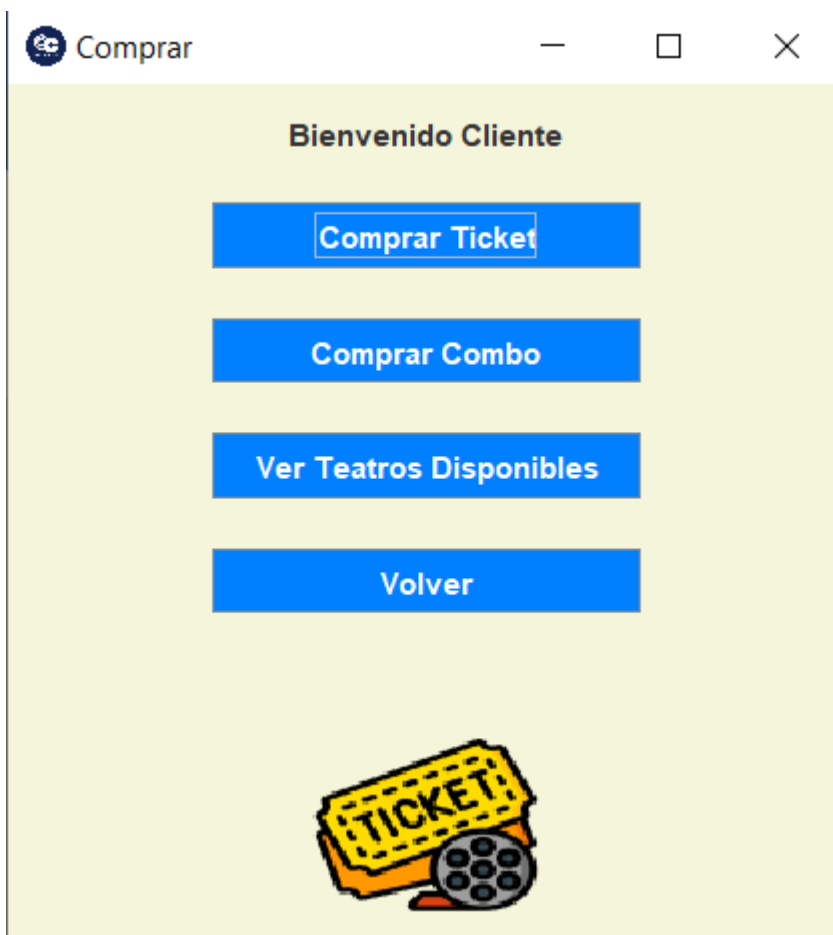
Agregar Combo

Eliminar Pelicula

Volver



Menú Comprar:



5. Conclusiones

En conclusión, el prototipo de software desarrollado por nuestro equipo para optimizar la gestión de turnos y venta de boletas en las salas de cine de Colombia representa un avance significativo e innovador en la transformación digital de la industria cinematográfica del país. Este proyecto, abordado metódicamente en tres fases, ha logrado abordar de manera integral los principales desafíos que enfrentan las empresas de cine colombianas.

En la primera fase, sentamos las bases sólidas del sistema mediante la implementación de una lógica y la programación orientada a objetos, definiendo las clases y algoritmos necesarios para manejar eficientemente la asignación de asientos y la gestión de turnos. La segunda fase se centró en brindar una experiencia de usuario intuitiva y atractiva a través de interfaces gráficas amigables, permitiendo a los clientes interactuar de forma fluida con el sistema y completar sus compras de manera rápida y sencilla. Finalmente, en la tercera fase, integramos una base de datos MySQL para la gestión centralizada de la información, garantizando la integridad de los datos y optimizando los procesos internos.

El resultado de nuestro esfuerzo colaborativo es un software escalable, fácil de usar y centrado en el usuario, que aborda efectivamente los problemas de confusión en los turnos y largas filas, mejorando así la experiencia de los espectadores. Al mismo tiempo, al aumentar la eficiencia operativa, nuestro sistema contribuye a mejorar la rentabilidad y productividad de las salas de cine, permitiéndoles mantenerse competitivas en un mercado en constante evolución.

Nuestro prototipo de software no solo optimiza la gestión de turnos y venta de boletas, sino que también representa un cambio hacia la transformación digital del sector del entretenimiento cinematográfico en Colombia, ofreciendo una solución innovadora y eficiente para un problema recurrente. Con su implementación, las salas de cine podrán brindar una experiencia más satisfactoria a sus clientes, al tiempo que optimizan sus operaciones internas gracias al arduo trabajo en equipo de nuestro grupo de desarrollo.

Referencias

- Universidad Europea. (2022, 24 agosto). Programación orientada a objetos. <https://universidadeuropea.com/blog/programacion-orientada-objetos/#:~:text=La%20programaci%C3%B3n%20orientada%20a%20objetos%20es%20un%20modelo%20de%20programaci%C3%B3n,l%C3%B3gica%20necesaria%20para%20esa%20manipulaci%C3%B3n.>
- Studocu. (s. f.). Temario fundamentos de bases de datos - Secretaría Académica, de Investigación e Innovación - Studocu. <https://www.studocu.com/es-mx/document/instituto-tecnologico-superior-de-alamo-temapache/fundamentos-de-base-de-datos/temario-fundamentos-de-bases-de-datos/35066007>
- Mkt, G. (2023, 30 junio). Sistema de gestión de Cine TMS-2000 | GDC Technology [Vídeo]. GDC Technology. <https://www.gdc-tech.com/soluciones/software-empresarial-de-cine/sistema-de-gestion-de-teatro-tms-2000/?lang=es>
- Prototipo de software - FasterCapital. (s. f.). FasterCapital. <https://fastercapital.com/es/contenido/Prototipo-de-software.html>
- Statista. (2023, 15 octubre). Colombia: número de asistentes a los cines 2000-2022. <https://es.statista.com/estadisticas/1199952/asistentes-boletos-cine-colombia/>



Facultad de ingeniería
Programa: Ingeniería de sistemas
Docente: Nombre omitido por privacidad
Asignatura: Diseño y Programación Orientada a Objetos

